Finite State Machines

ECE 2564, Embedded Systems

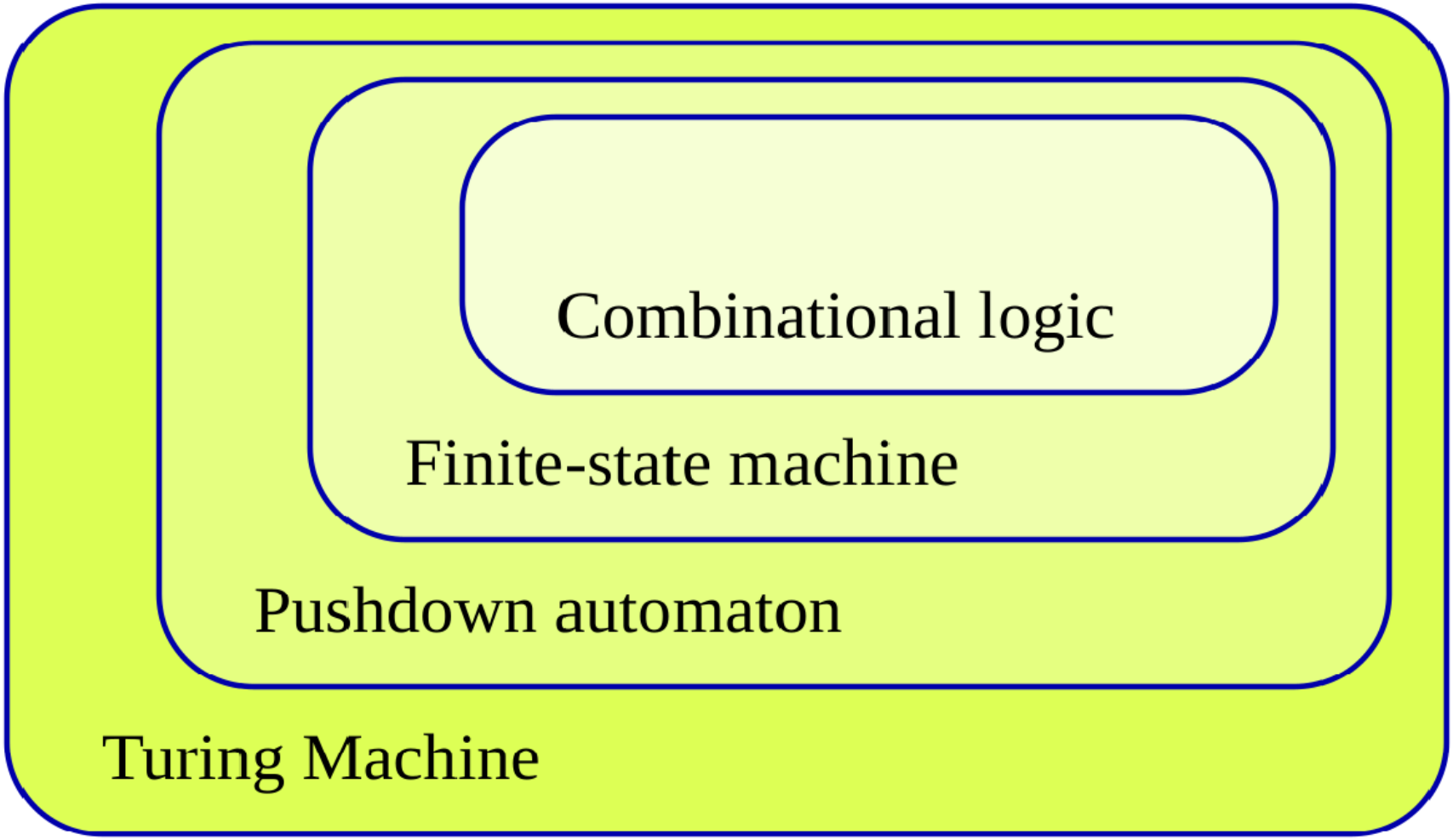
Fall 2025

Modeling Computation Problems

- An *abstract machine* is a theoretical model that allows for a detailed and precise analysis of how a computer system functions
- An *automaton* is an abstract self-propelled computing device which follows a predetermined sequence of operations automatically
- A *finite state machice (FSM)* is an automaton with a finite number of states
- *Automata theory* is the study of abstract machines and automata (plural of automaton)

Abstract Machine Classification

There are 4 major abstract machines



Automata Theory

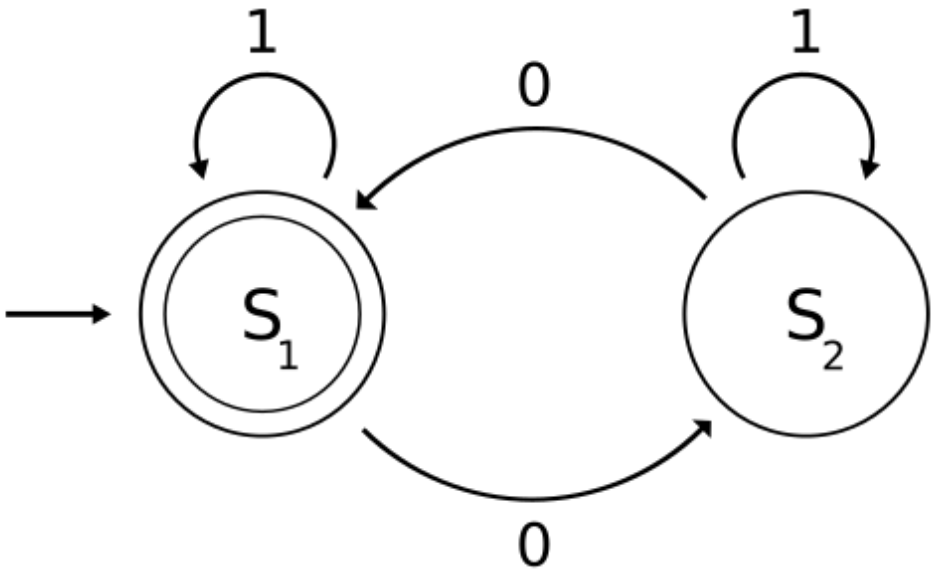
- Combinational logic
  - The output is a function only of the current combination of input
  - No internal states. No memory. (Combinational Logic)
- Finite-state Machine (We'll use this abstract machine)
  - The output and next state of the system is a function of the system's present state and a set of inputs. (Sequential Logic)
  - Finite number of states.
- Pushdown automaton
  - In addition to FSM, it employs a stack.
  - Use the top of the stack to decide which transition to take.
- TuringMachine
  - Operates on an infinite memory tape

Why FSM?

- Many problems can be modeled and solved as FSMs
- Using a FSM is a common approach for structuring hardware or sofware control
  - Most digital hardware control units are implemented as FSM
  - Embedded control software and device drivers are usually structured as FSMs

State Diagrams

- A *state diagram* can be used to represent a FSM



- The state diagram shows the relationships between the FSM's states, input signal and output signals
- What does this FSM do?

A Formal Description of a FSM

- A finite, non-empty set of possible *states* S
- A finite, non-empty set of possible *inputs* I
- A finite, possibly empty set of *outputs* O
- An *initial state* $s_0 \in S$
- A *state-transition function* $f : S \times I \longrightarrow S$
- An *output-function* $h : S \times I \longrightarrow O$

4-Digit ATM PIN Acceptor FSM Implementation[1]

- You are entering your 4-digit pin at an ATM to access your account
- As you enter your PIN, asterisks are displayed for each of the digits you enter
- After you enter the fourth digit
  - If you entered the correct PIN you get into your account
  - If you entered an incorrect PIN, the asterisks are cleared and you start over

[1] A real ATM PIN acceptor is more complicated.

ATM PIN Acceptor FSM

- Passcode: XXXX
- Input: guess $\in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- Output: *, **, ***, ****
- States: S0; S1,g_1; S2,$g_1 g_2$; S3,$g_1 g_2 g_3$; S4,$g_1 g_2 g_3 g_4$; InAccount

ATM PIN Acceptor FSM State Diagram

High-Level FSM Implementation in C

- Initialze
- Repeat:
  - Get an input value
  - Update state of FSM
  - Generate output

More Detailed Steps in Coding a FSM in C

- Specify states, inputs and outputs
- Draw state diagram or state table
- Define an `enum` for the states
- Define and `enum` for inputs and an `enum` for outputs, if needed
- Use a `switch` statement for processing the states
  - Every state must have a separate `case`
  - Remember to include a `break` for every case

Example FSM code

Summary

- A FSM was defined with a formal description
- Examples of state diagrams were provided
- The structure of code for a FSM was given